

26年算法设计与分析

1.分治算法 10

主定理

$$T(n) = aT(n/b) + f(n)$$

a:分解后的子问题

b:子问题的规模是原问题的1/b

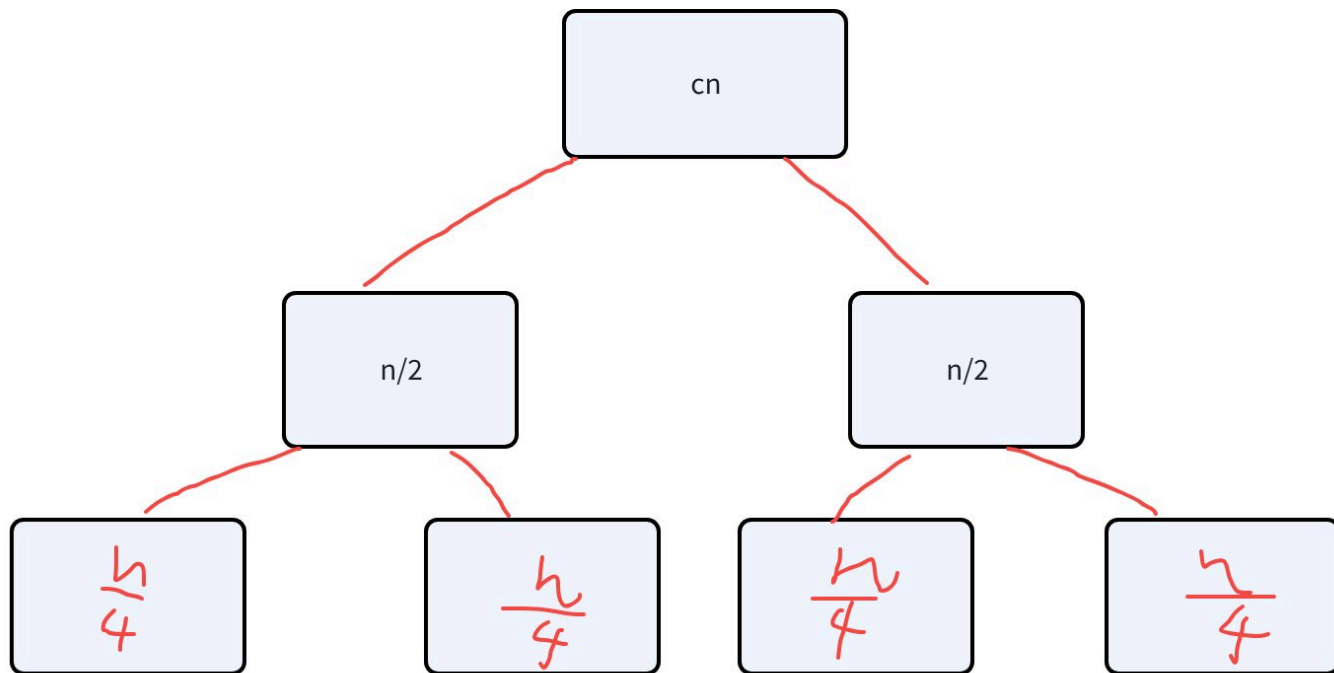
f(n):分解问题+合并子问题解的时间开销

本质是比较 $f(n)$ 和 $n^{\log_b a}$

$f(n)$	$n^{\log_b a}$	谁主导	$T(n) = ?$
slow	fast	子问题主导	$n^{\log_b a}$
速度相当		两者平衡	$n^{\log_b a} \log n$
fast	slow	$f(n)$ 主导	$f(n)$

递归树

e.g.归并排序 $T(n) = 2T(n/2) + cn$,画递归树



第0层: 根节点

第一层: 2个节点 $c(n/2)$

第二层: 4个节点 $c(n/4)$

.....

第k层: 2^k 个节点 $c(n/2^k)$, 总时间 cn

树高度 $n/2^k = 1$ 时, $k = \log_2^n$, 共 $\log_2^n + 1$ 层

总时间 $cn \log_2^n = O(n \log n)$

2. 动态规划 10

① 矩阵链乘法(区间DP)

给定矩阵序列, 不同乘法顺序会导致乘法次数差异极大, 求最小乘法次数

代码块

```
1  int matrix(vector<int>&p){
2      int n= p.size()-1;
3      vector<vector<int>> dp(n+1,vector<int>(n+1,0));
4      for(int l=2;l<=n;l++){
5          for(int i = 1;i<=n-l+1;i++){
6              int j=i+l-1;
```

```

7         dp[i][j]=INT_MAX;
8         for(int k=i;k<j;k++){
9             dp[i][j] = min(dp[i][j],dp[i][k]+dp[k+1][j]+p[i-1]*p[k]*p[j]);
10        }
11    }
12 }
13 return dp[i][n];
14 }

```

🏐 外层:枚举区间长度l

内层:枚举区间起点i

内层:枚举分割点k



状态转移方程: $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k+1][j] + p[i-1] * p[k] * p[j]);$

左矩阵行*中间公共列*右矩阵列

长度l从2开始,1个矩阵不用计算,终点 $j=i+l-1$,起点上限 $i \leq n-l+1$

②背包问题

1: 01背包（每件最多选1次）

给定 n 个物品，每个物品重量 $w[i]$ 、价值 $v[i]$ ，背包最大承重为**阈值** V 。每件物品只能选或不选，总重量 $\leq V$ ，求能获得的最大总价值。

DP状态

$dp[j]$ ：容量 j 下的最大价值

核心规则

- 一维优化、**倒序**遍历容量
- 时间： $O(nV)$

代码块

```

1  int bag01(int n, int V, vector<int>& w, vector<int>& v)
2  {
3      vector<int> dp(V + 1, 0);
4      for(int i = 0; i < n; i++)
5      {
6          // 倒序，防止重复选取

```

```

7         for(int j = V; j >= w[i]; j--)
8         {
9             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
10        }
11    }
12    return dp[V];
13 }

```

易错考点

1. 倒序遍历 = 每件只能选1次
2. 初始dp全0：允许总重量小于阈值

2：完全背包（每件无限任选）

物品不限次数选取，总重量不超过阈值，求最大价值。

核心规则

正序遍历容量

代码块

```

1  int bagFull(int n, int V, vector<int>& w, vector<int>& v)
2  {
3      vector<int> dp(V + 1, 0);
4      for(int i = 0; i < n; i++)
5      {
6          // 正序，可以重复拿
7          for(int j = w[i]; j <= V; j++)
8          {
9              dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
10         }
11     }
12     return dp[V];
13 }

```

3.贪心问题 15

①分数背包

有n种物品和1个容量为c的背包,第i种物品的重量为w[i],价值为v[i],物品可以分割,目标是让背包中物品的总价值最大

$value_per_weight[i] = v[i] / w[i]$

桂林电子科技大学

贪心①

实验报告

实验名称 分数背包

辅导员意见:

系 _____ 专业 _____

班第 _____ 实验小组 _____

作者 _____ 学号 _____

同作者 _____

实验日期 _____ 年 _____ 月 _____ 日

成绩

辅导员
签名

```
bool cmp(pair<double, double> a, pair<double, double> b)
    return a.second/a.first > b.second/b.first;

int main() {
    double c = 50; //背包容量
    vector<pair<double, double>> v = {{10, 60}, {20, 100}, {30, 120}};
    sort(v.begin(), v.end(), cmp);
```

```
double res = 0;
```

```
for (auto x : v) {
```

```
    if (c <= 0) break; //装满, 停止
```

```
    if (x.first <= c) { //物品能全装下
```

```
        res += x.second; //全拿
```

```
        c -= x.first;
```

```
    } else { //装一部分
```

```
        res += c * x.second / x.first;
```

```
        c = 0;
```

```
    }
```

```
}
```

```
cout << res << endl; //最大价值
return 0;
```

②区间任务调度

服务器调度...(小根堆维护结束时间)

桂林电子科技大学

贪心②

实验报告

实验名称 区间任务调度

系 _____ 专业 _____

班第 _____ 实验小组 _____

作者 _____ 学号 _____

同作者 _____

实验日期 _____ 年 _____ 月 _____ 日

辅导员意见:

成绩

辅导员
签名

```
bool cmp(pair<int, int> a, pair<int, int> b)
    return a.second < b.second;

int main() {
    vector<pair<int, int>> v = {{1, 2}, {3, 4}, {0, 6}, {5, 7}, {8, 9}};
    sort(v.begin(), v.end(), cmp);
    int cnt = 1; // 至少选一个活动
    int last = v[0].second; // 记录上一个结束时间
    for (int i = 1; i < v.size(); i++) {
        if (v[i].first >= last) { // 下个活动开始时间 >= 上一个结束时间
            cnt++; // 活动数+1
            last = v[i].second; // 更新结束时间
        }
    }
    cout << cnt << endl; // 输出最多选几个
    return 0;
}
```



4.算法设计 50

4.1约束最大化问题 25


```

}
int main () {
    scanf ("%d%d", &n, &q);
    for (int i = 1; i < n; ++i) {
        int x; scanf ("%d", &x);
        int b = son[x][0] > 0;
        scanf ("%d%d", &son[x][b], &val[x][b]);
    }
    dfs (1);
    printf ("%d\n", dp[1][q]);
    return 0;
}

```

例 16-3 选课(洛谷 P2014, CTSC 1997)。

给定 n 门课程来选, 每门课程 i 有依赖的课程 k_i , 即选了 k_i 才能选 i , 如果 $k_i = 0$ 则表示没有依赖。每门课程有一个权值 s_i , 选出 m 门课, 要求最大化被选的课程权值和。

数据范围: $n, m \leq 300, s_i \leq 20$ 。

分析: 将 k_i 向 i 连一条有向边, 得到的图由一棵根为 0 的外向树和若干环构成。环是没有意义的, 于是问题被转化成在树上选择若干个结点, 最大化权值。因此可以考虑树形 DP。考虑子树作为子结构的时候, 无论怎么选, 只有“子树中一共选了多少课程”这一条件能够影响其到父结点的转移。

根据先前的经验, 很容易写出 DP 状态: $f_{u,w}$ 表示根为 u 的子树中, 选择 w 门课程, 权值和最大是多少。

在转移到父结点的时候, 对于每个子树枚举其选择了多少课程。直接枚举的话复杂度是错误的, 简单分析发现这是一个分组背包模型(即物品有若干类, 同一类里面恰好只能选一个物品的背包模型)。

在 u 为根的子树中, 先假设 u 不选。用 $g_{v,w}$ 来辅助背包的转移, 表示当前考虑到子树 v , 一共选了 w 门课程, 权值和最大是多少。在枚举到下一个子树 v' 的时候有转移:

$$g_{v', w+w'} = \max_{0 \leq w+w' \leq m} \{g_{v,w} + f_{v',w'}\}$$

然后再考虑 u 怎么选的情况。当子树中选了 0 门课程, u 可以选, 也可以不选。一旦子树中选择了大于 0 个, u 就不得不选了。记 v 为最后一个枚举到的子树, 有转移:

$$f_{u,0} = g_{v,0}$$

$$f_{u,w} = g_{v,w-1} + s_u \quad (w > 0)$$

至此完成了一个较为复杂的树上 DP。

...的DP。代码如下:

```
#include <bits/stdc++.h>

using namespace std;

const int MAXN = 310;

int f[MAXN][MAXN], n, m;
int req[MAXN], scr[MAXN];
```



```
int head[MAXN], nxt[MAXN];
void adde (int b, int e) {
    nxt[e] = head[b]; head[b] = e;
}

void dfs (int u) {
    memset (f[u] + 1, 0xc0, (m + 1) << 2);
    for (int v = head[u]; v; v = nxt[v]) {
        dfs (v);
        static int g[MAXN]; //使用滚动数组
        memset (g, 0xc0, (m + 1) << 2);
        for (int j = 0; j <= m; ++j)
            for (int k = 0; k <= m; ++k) {
                //对子树进行分组背包
                g[j + k] = max (g[j + k], f[u][j] + f[v][k]);
            }
        memcpy (f[u], g, (m + 1) << 2);
    }
    if (u == 0) return;
    //考虑根的选择与不选
    for (int i = m; i; --i)
        f[u][i] = f[u][i - 1] + scr[u];
}

int main () {
    scanf ("%d%d", &n, &m);
    for (int i = 1; i <= n; ++i) {
        scanf ("%d%d", &req[i], &scr[i]);
        adde (req[i], i);
    }
    dfs (0);
    printf ("%d\n", f[0][m]);
}
```



4.2冲突矩阵处理 25

回溯dfs模板

```
1 void dfs(int k){ //k代表递归层数
2     if(所有空已经填完){
3         判断最优解/记录答案;
4         return;
5     }
6
7     for(枚举这个空能填的选项){
8         if(这个选项合法){
9             记录这个空(保存);
10            dfs(k+1);
11            取消这个空(恢复);
12        }
13    }
14 }
```

e.g.8皇后

代码块

```
1 void dfs(int x) {
2     if (x > n) {
3         ans++;
4         //输出
5         return;
6     }
7     for (int i = 1; i <= n; i++) {
8         if (b1[i] == 0 && b2[x+i] == 0 && b3[x-i+15] == 0) {
9             a[x] = i;
10            b1[i] = 1; b2[x + i] = 1; b3[x - i + 15] = 1;
11            dfs(x + 1);
12            b1[i] = 0; b2[x + i] = 0; b3[x - i + 15] = 0;
13        }
14    }
```

```
14     }
15 }
```

NP难问题/NP完全问题

P 问题能在**多项式时间**内直接求出答案，求解很快。例：排序、最短路、二分查找。

NP 问题找解很难，但给一个候选答案，能**多项式时间快速验证**对错。八皇后、数独、旅行商都属于这类。

NP 完全 (NPC)NP 里最难的一类；**所有 NP 问题都可以规约转化成它**，只要找到它的多项式解法，所有 NP 问题都能快速求解。n 皇后存在性判定就是经典 NPC。

e.g.细菌

5. 一矩形阵列由数字 0 到 9 组成,数字 1 到 9 代表细菌,一个细菌的定义为沿细菌数字上下左右还是数字则为同一个细菌,请设计算法求给定矩形阵列的细菌个数。如:

4 行 10 列的如下数字矩阵:

0234500067	注释: 第一行的末尾 67 是同一个, 第 2 行开头的数字 1 和第三行开头的数字 2
1034560500	是同一个, 第一行中间的 2345 和第二行中间的 3456 及第三行中间的 456 由于存
2045600671	在上下左右关系, 都是同一个, 剩下的第二行的靠右端的数字 5 和第三行末端的
0000000089	671 和第四行末端的 89, 也是同一个。因此共有 4 个不同的细菌。

- (1) (5 分) 用文字描述你进行设计的算法;
- (2) (15 分) 设计算法实现题目要求。
- (3) (5 分) 该问题是属于 NP 难度问题么? 分析你采用的算法的时间复杂度。

代码块

```
1  vector<vector<int>> grid;
2  vector<vector<bool>> visited;
3  int rows, cols;
4
5  int dirs[4][2] = {{-1,0}, {1,0}, {0,-1}, {0,1}}; // 四个方向: 上下左右
6
7  void dfs(int k){
8      int x = k / cols;
9      int y = k % cols;
10     if (x < 0 || x >= rows || y < 0 || y >= cols || visited[x][y] || grid[x]
        [y] == 0) {
11         return;
12     }
13     for (int i = 0; i < 4; ++i) {
14         int nx = x + dirs[i][0];
```



```

15         int ny = y + dirs[i][1];
16         int new_k = nx * cols + ny;
17         if (nx >= 0 && nx < rows && ny >= 0 && ny < cols && !visited[nx][ny]
    && grid[nx][ny] != 0) {
18             visited[nx][ny] = true;
19             dfs(new_k);
20             // 细菌计数无需恢复
21         }
22     }
23 }

```

5.算法优化 15

最长连续子数组乘积最大值答案

```

1  class Solution {
2  public:
3      int maxProduct(vector<int>& nums) {
4          if (nums.empty()) return 0;
5
6          int maxn = nums[0];
7          int minn = nums[0];
8          int res = nums[0];
9          for(int i =1 ; i<nums.size();i++){
10             int tempMax=max({nums[i],maxn*nums[i],minn*nums[i]});
11             int tempMin=min({nums[i],minn*nums[i],maxn*nums[i]});
12
13             maxn = tempMax;
14             minn = tempMin;
15             res = max(res, maxn);
16         }
17         return res;
18     }
19 };

```